StyleRadar

Descubre tu estilo. En tu ciudad.

DOCUMENTO DE ARQUITECTURA Y DISEÑO DEL FRONTEND

Arquitectura propuesta de nivel 3 · agrupación por funcionalidades/módulos

Desarrollo y Operaciones de Software (DOSW) | Sprint 4

Escuela Colombiana de Ingeniería Julio Garavito

Equipo: Juan Nicolás Álvarez · Camilo Ortiz · Daniel Valero · Juan Diego Valderrama · Paula Alejandra Díaz

Versión 1.1 · Octubre de 2026 · Estado: propuesta de diseño para revisión

IDENTIDAD El documento emplea la paleta corporativa del repositorio. La tipografía oficial definida es Poppins; este archivo usa una alternativa de lectura para mantener portabilidad y edición.

Control del documento

Campo	Valor
Proyecto	StyleRadar
Artefacto	High Level Design — Frontend
Curso	Desarrollo y Operaciones de Software (DOSW)
Sprint	4 — cierre, documentación y feria
Versión	1.1, propuesta para revisión
Fuente de diseño	Frontend-Style-Radar / docs/sprint4-arquitectura-frontend / styleradar-frontend-v3 (prototipo heredado de main)
Fuente de requisitos	Análisis de 75 RF y guía oficial Sprint 4
Estado de implementación	Prototipo HTML v3 simulado; React + API no implementado en esta entrega

Resumen ejecutivo

StyleRadar es una plataforma de descubrimiento de moda local en Bogotá. Permite explorar prendas y almacenes cercanos, consultar disponibilidad y tallas, componer playlists de estilo, probar visualizaciones aproximadas y gestionar la segunda vida de la ropa mediante venta o donación. Sus actores son comprador, administrador de almacén, representante de fundación y administrador de la plataforma.

Este documento describe la experiencia y la arquitectura prevista del frontend con enfoque modular de nivel 3. Identifica las pantallas y los permisos por rol, los flujos normales y alternos, el diseño de interacción con la API REST del backend, las decisiones de identidad, accesibilidad, pruebas y las limitaciones conocidas. No afirma que exista una aplicación React conectada: el entregable actual sigue siendo un prototipo HTML navegable.

Índice de contenidos

01. Introducción, propósito y alcance
02. Contexto y actores
03. Vista general de la solución
04. Arquitectura nivel 3 y módulos
05. Tecnologías y decisiones de diseño
06. Pantallas y accesos por rol
07. Navegación y flujos de interacción
08. Funciones y recorridos por módulo
09. Diseño de integración con backend
10. Seguridad, errores y estados
11. Identidad visual y accesibilidad
12. Validación, trazabilidad y limitaciones
13. Referencias y revisión

1. Introducción, propósito y alcance

1.1. Descripción de la solución

StyleRadar conecta al usuario con oferta de ropa de almacenes locales. El portal presenta el inventario y las prendas disponibles en la ciudad, apoya el descubrimiento geográfico y propone experiencias de estilo personalizadas. El producto no se plantea como un marketplace de compra de prendas: favorece el encuentro entre personas, prendas y tiendas físicas.

1.2. Propósito del documento

Definir una arquitectura mantenible que permita implementar posteriormente la experiencia frontend, documentar el comportamiento esperado por rol y preservar la coherencia con el prototipo y los contratos publicados por el backend.

1.3. Alcance de Sprint 4

Incluido	Fuera del alcance de esta entrega
Diseño de arquitectura, carpetas y responsabilidades por módulo.	Crear el proyecto React productivo o integrar endpoints en ejecución.
Matrices de pantallas/roles y flujos normales, de error y vacíos.	Publicar un sitio React funcional o anunciar una URL de despliegue que no existe.
Tecnologías previstas y contratos de comunicación ilustrativos.	Implementar las integraciones con Axios o cambiar el backend.
Análisis de diseño visual, prototipo HTML y trazabilidad.	Afirmar que el HTML interactivo es una integración real con Spring Boot.

DISTINCIÓN La guía oficial del Sprint 4 exige documentación, prototipo y README del frontend; señala que no se debe subir aún código React al repositorio frontend. El repositorio recibido ya contiene un prototipo HTML v3 que se registra como artefacto previo.

1.4. Objetivos de calidad

- Modularidad: cada función encapsula sus pantallas, hooks, servicios, estado y validaciones de presentación.
- Trazabilidad: conservar ID de RF/HU y relacionar cada pantalla con requerimientos y contratos del backend.
- Seguridad: autorización en el frontend para UX, siempre complementada por controles efectivos del backend.
- Usabilidad y accesibilidad: comportamiento responsive, lenguaje de errores comprensible y estados vacíos útiles.
- Evolución: sustituir datos simulados por servicios reales sin reescribir toda la interfaz.

2. Contexto y actores

2.1. Usuarios y responsabilidades

Actor en interfaz	Rol backend correspondiente	Responsabilidades principales
Visitante anónimo	Sin sesión	Explorar portada, catálogo público, mapa, tiendas y contenido informativo.

Actor en interfaz	Rol backend correspondiente	Responsabilidades principales
Comprador	COMPRADOR	Consultar productos, perfil, preferencias, playlists, alertas, segunda vida y donaciones.
Administrador de almacén	ADMIN_ALMACEN	Gestionar catálogo, tienda, colecciones, suscripción y solicitudes.
Representante de fundación	REPRESENTANTE_FUNDACION	Gestionar perfil de fundación y solicitudes de donación.
Administrador StyleRadar	ADMIN_STYLERADAR	Administrar usuarios, moderación, verificación y demás funciones autorizadas.

Los nombres exactos de los cuatro roles de backend se verificaron en el enum Rol.java de StyleRadar-Backend. Las pantallas descritas proceden del README y de los mockups del repositorio frontend. Que una pantalla figure en el prototipo no implica que todas sus operaciones dispongan ya de endpoint.

2.2. Límites del sistema

El cliente web muestra el estado de la sesión, procesa interacción y presenta resultados. El backend de Spring Boot controla autenticación, autorización, reglas de negocio, persistencia y respuestas HTTP. En la arquitectura futura el frontend no accede directamente a PostgreSQL; se comunica exclusivamente con servicios REST autenticados o públicos.

3. Vista general de la solución

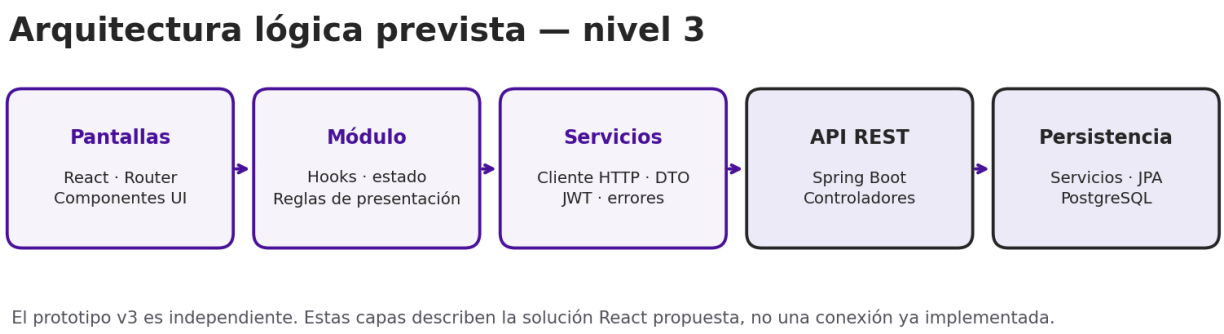


Figura 1. Flujo técnico propuesto: interfaz y módulos React → servicios HTTP → API REST → persistencia.

3.1. Componentes y responsabilidades

Componente	Responsabilidad prevista
Presentación	Pantallas, layouts, formularios, estados loading/empty/error y elementos accesibles.
Router	Resolver rutas, proteger navegación por rol y dirigir al inicio de sesión si corresponde.
Módulos	Agrupar lógica y visuales de un dominio: auth, catálogo, mapa, segunda vida, etc.
Servicios de integración	Enviar solicitudes HTTP y adaptar DTO de respuesta a modelos de vista.
Core compartido	Cliente HTTP, configuración, diseño corporativo, componentes comunes y utilidades.
Spring Boot	Validar identidad y reglas del negocio; responder con datos o ErrorResponseDTO.

4. Arquitectura nivel 3: organización modular

4.1. Justificación de la elección

La presentación de DOSW 1, semana 11, distingue tres formas de organización. El nivel 3 se agrupa por funcionalidades o módulos y define límites claros para grandes sistemas. Para StyleRadar evita distribuir archivos de una misma función entre carpetas globales, facilita el trabajo paralelo y permite que cada módulo evolucione con sus propias reglas.

4.2. Estructura de directorios propuesta

```
src/
├── assets/                  # medios e imágenes
├── modules/
│   ├── core/                # design-system, config, rutas, HTTP
│   │   ├── components/
│   │   ├── design-system/
│   │   ├── hooks/
│   │   ├── lib/
│   │   └── utils/
│   ├── auth/                # sesión, login y registro
│   ├── discovery/           # portada, categorías y exploración
│   ├── catalogo/            # búsqueda, filtros y ficha de prenda
│   ├── almacenes/           # mapa, tienda pública y panel de inventario
│   ├── probador/            # visualización aproximada del look
│   ├── playlists/           # colecciones y looks
│   ├── preferencias/        # perfil, tallas, wishlist y recomendaciones
│   ├── alertas/             # búsquedas guardadas y avisos
│   ├── segunda-vida/        # venta, donación y publicaciones
│   ├── mensajes/            # mensajería entre actores
│   ├── suscripciones/       # planes y renovación; consulta de pagos
│   ├── fundaciones/         # perfil público y solicitudes
│   └── administracion/      # usuarios, verificación, moderación
└── app/                     # composición y arranque futuro
```

Dentro de cada módulo, y solo si la complejidad lo requiere, se usarían subcarpetas components/, pages/, hooks/, services/, states/, models/ y utils/. Esta organización es una adaptación al dominio StyleRadar, no una estructura encontrada actualmente en el repositorio.

4.3. Contrato interno de un módulo

Carpeta	Regla de responsabilidad
components/	Elementos propios de la funcionalidad; no ejecutan directamente peticiones HTTP.
pages/	Pantallas y composición de componentes de una ruta.
hooks/	Coordinar interacciones, recuperación de datos y estados de la pantalla.
services/	Consumir endpoints, serializar filtros y traducir respuestas o errores.
models/	Tipado local y contratos DTO sin acoplar la UI a las entidades del backend.
states/	Estado global del dominio cuando es necesario compartirlo.
utils/	Funciones puras específicas del dominio.

4.4. Reglas de dependencia

1. Cada pantalla consume hooks o servicios de su propio módulo y componentes del core.
2. Los componentes del core no importan módulos del negocio.
3. Los módulos no acceden a la base de datos ni a repositorios JPA.
4. Los servicios no manipulan directamente el DOM.
5. Una funcionalidad compartida se publica mediante interfaz explícita; no se accede a archivos internos de otros módulos sin contrato.
6. Los DTO y rutas se cotejan con OpenAPI/Swagger antes de implementar cualquier llamada real.

4.5. Correspondencia funcional y módulos adicionales

La distribución de nivel 3 compartida con el README conserva core, auth, catalogo, almacenes, playlists, alertas, preferencias, suscripciones, segunda-vida, fundaciones y administracion. Core es soporte técnico transversal; los demás agrupan funcionalidades. Preferencias reúne el perfil (SR-RF-03–06). Almacenes incluye tanto la ficha pública y el mapa (SR-RF-17–22) como Mi tienda, inventario y fotos (SR-RF-50–60 y SR-RF-74); panel-almacen pasa a ser una agrupación interna de almacenes, sin eliminar pantallas.

Suscripciones concentra planes y renovación (SR-RF-67–68), visibles en #/cuenta/suscripcion. La administración compone la consulta de pagos (SR-RF-75) mediante su interfaz pública y conserva los permisos administrativos. Los planes del comprador son una pantalla del HTML cuya contratación y soporte REST deben confirmarse; no se atribuyen a los RF de suscripción de almacenes.

Se justifican tres módulos adicionales presentes en el diseño original: discovery compone la portada #inicio, categorías y accesos a catálogo, tiendas, fundaciones y playlists; reutiliza SR-RF-07–22, SR-RF-43 y SR-RF-69–72 sin inventar un requisito. Probador encapsula la selección de prendas, foto y visualización aproximada de #/probador (SR-RF-23–27). Mensajes concentra #/mensajes para compradores, almacenes y fundaciones (SR-RF-40–41, SR-RF-48 y SR-RF-73), evitando duplicar conversaciones dentro de segunda-vida.

Los nombres perfil y panel-almacen del diseño previo designan ahora agrupaciones funcionales internas de preferencias y almacenes. Se conservan sus recorridos en las secciones 8.5 y 8.11. Catálogo y playlists abren el probador por su interfaz pública; segunda-vida, almacenes y fundaciones usan mensajes. Esta revisión coteja responsabilidades con el HTML real y los 75 RF del Excel v3; no certifica ejecución E2E ni contratos del backend.

5. Tecnologías y decisiones de diseño

Tecnología	Estado	Uso y justificación
HTML/CSS/JavaScript	Existente en prototipo v3	Prototipo navegable monolítico simulado, con almacenamiento local de pruebas.
React + JSX	Propuesta	Interfaz basada en componentes y flujo declarativo.
Vite	Propuesta	Servidor y empaquetador del proyecto React.
React Router	Propuesta	SPA, rutas por rol, navegación y rutas protegidas.
Axios (o fetch)	Propuesta	Cliente HTTP centralizado para consumir API REST, bearer token y errores.
Context API / hooks	Propuesta	Sesión y estados transversales sin duplicar estado.
CSS / CSS Modules	Propuesta	Estilos locales y tokens del sistema visual.

Tecnología	Estado	Uso y justificación
Prototipado navegable	Prototipo HTML v3 existente; no se utilizó Figma	El equipo no dispone de un diseño en Figma. El prototipo interactivo se encuentra en styleradar-frontend-v3/prototipo/index.html, en el repositorio GitHub. Para interactuar, descargar el repositorio y abrir el HTML en el navegador. La guía del profesor solicita explícitamente Figma: confirmar si acepta este sustituto.
Spring Boot REST	Backend existente	Origen de las reglas de negocio y contratos HTTP; no se sustituye en el frontend.
Azure / Vercel	No se declara despliegue frontend	Azure corresponde al backend; hosting frontend se definiría en implementación posterior.

6. Pantallas y acceso por rol

La matriz expresa acceso de navegación previsto a partir de los mockups. P = público, S = permitido por rol, — = no corresponde. La autorización final de operaciones la impone siempre la API.

Pantalla / grupo	Invitado	Comprador	Almacén	Fundación	Admin
Portada / Cómo funciona	P	S	S	S	S
Login / Registro	P	P	P	P	P
Catálogo, detalle y mapa	P	S	S	S	S
Perfil, búsquedas, alertas	—	S	—	—	—
Probador / playlists personales	—	S	—	—	—
Segunda vida, publicar / donar	—	S	—	—	—
Mensajes	—	S	S	S	—
Mi tienda / inventario	—	—	S	—	—
Suscripción de almacén	—	—	S	—	—
Mi fundación / donaciones	—	—	—	S	—
Administración / moderación	—	—	—	—	S
Gestión de usuarios y verificación	—	—	—	—	S

NOTA Algunas pantallas pueden ser consultables sin autenticación si los endpoints públicos lo permiten. Acciones personales, administrativas y de modificación requieren sesión y permisos correspondientes.



Figura 2. Ramas principales de navegación por tipo de actor.

7. Navegación y recorridos

7.1. Entrada y navegación general

Desde la portada, un visitante puede recorrer categorías, buscar ropa, consultar el mapa y entrar a un almacén. Al intentar un recurso protegido se muestra un mensaje que indica qué exige autenticación y se ofrece iniciar sesión. Una sesión válida determina el menú y las rutas disponibles para cada rol.

7.2. Recorrido del comprador



Figura 3. Happy path del descubrimiento y personalización.

7.3. Flujos de los demás actores

Rol	Secuencia funcional de navegación
Administrador de almacén	Ingresar → Mi tienda → Agregar prenda (3 pasos) → enviar a revisión → catálogo / estadísticas → gestionar suscripción.
Representante de fundación	Ingresar → Mi fundación → Solicitudes de donación → aceptar o rechazar → confirmar recepción.
Administrador StyleRadar	Ingresar → resumen → moderación / verificación / usuarios → revisar detalle → confirmar acción → ver estado actualizado.

8. Funcionalidades del sistema y comportamiento por módulo

Se describen aquí los módulos mostrados por el prototipo y su eventual traducción a React. Los flujos normales corresponden al diseño existente; la viabilidad de las operaciones contra API debe confirmarse mediante Swagger y el análisis actualizado de requerimientos. Las carpetas de nivel 3 son propuestas; el HTML mantiene estas funciones

dentro de un solo archivo. Los errores HTTP, la expiración JWT y la persistencia de los diagramas corresponden a la integración futura, no a llamadas reales del prototipo.

8.1. Autenticación y registro

Elemento	Descripción
Módulo de nivel 3	auth
RF/HU asociados	SR-RF-01 y SR-RF-02
Pantallas	Login, registro y entrada por rol.
Flujo básico	El usuario abre Inicio de sesión, suministra datos válidos, recibe autorización simulada en el prototipo y accede al menú de su rol.
Errores y estados alternos	Contraseña inválida: mensaje bajo el campo; cuenta suspendida o token inválido: acceso denegado; mostrar opción de corregir.
Mockups disponibles	assets/mockups/02-login.jpg, 03-registro.jpg

8.2. Inicio y descubrimiento

Elemento	Descripción
Módulo de nivel 3	discovery
RF/HU asociados	Composición de SR-RF-07–22, SR-RF-43 y SR-RF-69–72; sin RF nuevo
Pantallas	Portada, categorías, almacenes y fundaciones.
Flujo básico	Navega por categorías, selecciona una tienda y visualiza prendas y información del local.
Errores y estados alternos	Sin resultados: estado vacío con filtros alternativos; sin sesión al intentar acción personal: ofrecer login.
Mockups disponibles	assets/mockups/01-portada.jpg, 05-categorias.jpg, 06-tiendas.jpg

8.3. Catálogo y fichas

Elemento	Descripción
Módulo de nivel 3	catalogo
RF/HU asociados	SR-RF-07–16 y SR-RF-22
Pantallas	Buscador, filtros y detalle.
Flujo básico	El usuario busca por palabra, talla, color, estilo o precio; ordena y consulta la ficha y prendas similares.
Errores y estados alternos	Filtros sin coincidencias: proponer relajar criterios o guardar búsqueda; error del servidor: reintentar.
Mockups disponibles	assets/mockups/10-catalogo.jpg

8.4. Mapa y almacenes

Elemento	Descripción
Módulo de nivel 3	almacenes
RF/HU asociados	SR-RF-17–22; panel de inventario SR-RF-50–60 y fotos SR-RF-74
Pantallas	Mapa de Bogotá, distancia, catálogo resumido y novedades.
Flujo básico	El usuario consulta marcadores, filtra categoría, elige un almacén y ve su resumen y distancia.
Errores y estados alternos	Sin coordenadas: estado informativo; tienda inexistente 404; error de ubicación: elegir barrio manual.
Mockups disponibles	assets/mockups/07-mapa-inicio.jpg, 11-mapa.jpg, 12-tienda.jpg

8.5. Perfil y preferencias

Elemento	Descripción
Módulo de nivel 3	preferencias (perfil, tallas, wishlist y recomendaciones)
RF/HU asociados	SR-RF-03–06; wishlist y datos de cuenta visibles en el HTML
Pantallas	Mi cuenta, recomendaciones, wishlist y preferencias.
Flujo básico	Una sesión de comprador abre Mi cuenta y modifica sus preferencias para actualizar recomendaciones.
Errores y estados alternos	Sin recomendaciones: explicar cómo completar el perfil; datos inválidos: corrección por campo; 401: nueva autenticación.
Mockups disponibles	assets/mockups/18-perfil.jpg

8.6. Alertas y búsquedas guardadas

Elemento	Descripción
Módulo de nivel 3	alertas
RF/HU asociados	SR-RF-28–32
Pantallas	Mis búsquedas y alertas, notificaciones.
Flujo básico	El comprador guarda una búsqueda y configura alerta; cuando aparece coincidencia, el sistema presenta notificación.
Errores y estados alternos	Búsqueda sin resultados: sugerencias; error de registro: aviso de conflicto; sin alertas: CTA crear. Validación HTTP positiva pendiente de evidencia.
Mockups disponibles	assets/mockups/18-perfil.jpg (sección Mis búsquedas y alertas)

8.7. Probador virtual

Elemento	Descripción
Módulo de nivel 3	probador
RF/HU asociados	SR-RF-23–27
Pantallas	Maniquí, selección de prendas y foto de referencia.
Flujo básico	El comprador elige look y observa composición aproximada sobre un maniquí o ejemplo.
Errores y estados alternos	Foto inválida o no disponible: mensaje claro y opción de usar foto de ejemplo. No afirmar un modelo IA real conectado.
Mockups disponibles	assets/mockups/14-probador.jpg

8.8. Playlists de estilo

Elemento	Descripción
Módulo de nivel 3	playlists
RF/HU asociados	SR-RF-69–71; selección destacada en administración SR-RF-72
Pantallas	Listas de prendas, likes y selección de looks.
Flujo básico	Crear o consultar playlist, añadir prendas, guardar y probar un look.
Errores y estados alternos	Lista vacía: sugerencias; prenda agotada: etiqueta No disponible; restricción prevista hasta 15 prendas. Selección administrativa para destacar no está probada en backend.
Mockups disponibles	assets/mockups/15-playlist.jpg

8.9. Segunda vida y donaciones

Elemento	Descripción
Módulo de nivel 3	segunda-vida
RF/HU asociados	SR-RF-33–39 y SR-RF-42–45; coordinación SR-RF-48
Pantallas	Publicar usado, vender, donar, solicitudes.
Flujo básico	El comprador publica ropa usada o propone donación con modo, fecha, franja y punto; la fundación responde.
Errores y estados alternos	Falta foto o franja: validar; precio no permitido: explicar mínimo; solicitud rechazada: mostrar motivo.
Mockups disponibles	assets/mockups/09-segunda-vida.jpg, 16-vender.jpg, 25-donaciones.jpg

8.10. Mensajes

Elemento	Descripción
Módulo de nivel 3	mensajes

Elemento	Descripción
RF/HU asociados	SR-RF-40–41, SR-RF-48 y SR-RF-73
Pantallas	Bandeja, conversaciones y mensajes por actor.
Flujo básico	El usuario inicia conversación con tienda o vendedor desde ficha y consulta respuesta en mensajes.
Errores y estados alternos	Sin conversaciones: empty state; destinatario no disponible: mensaje y retorno. No atribuir endpoint sin verificar.
Mockups disponibles	assets/mockups/17-mensajes.jpg

8.11. Panel del almacén

Elemento	Descripción
Módulo de nivel 3	almacenes (panel e inventario), suscripciones (planes y renovación), mensajes (conversaciones)
RF/HU asociados	SR-RF-50–60, SR-RF-67–68, SR-RF-73 y SR-RF-74
Pantallas	Mi tienda, inventario, fotos, estadísticas, suscripción.
Flujo básico	El administrador añade prenda en tres pasos y gestiona plan; actualiza disponibilidad y revisa mensajes.
Errores y estados alternos	Sin fotos/unidades: no avanzar; suscripción duplicada: advertir conflicto; cancelación conserva período actual.
Mockups disponibles	assets/mockups/21-mi-tienda.jpg, 22-tienda-estadisticas.jpg, 23-tienda-suscripcion.jpg

8.12. Fundación aliada

Elemento	Descripción
Módulo de nivel 3	fundaciones
RF/HU asociados	SR-RF-43–49; gestión administrativa SR-RF-63–65
Pantallas	Página pública de fundación y solicitudes.
Flujo básico	La fundación recibe propuesta de donación, confirma disponibilidad y acepta o rechaza; registra recepción.
Errores y estados alternos	Rechazar sin motivo: exigirlo; cuenta inactiva: acceso restringido; bandeja vacía: información útil.
Mockups disponibles	assets/mockups/24-mi-fundacion.jpg, 25-donaciones.jpg

8.13. Administración de plataforma

Elemento	Descripción
Módulo de nivel 3	administracion
RF/HU asociados	SR-RF-61–66, SR-RF-72 y SR-RF-75
Pantallas	Usuarios, almacenes, moderación, fundaciones y pagos.

Elemento	Descripción
Flujo básico	Un administrador revisa solicitudes de verificación o moderación, registra decisión y consulta estado actualizado.
Errores y estados alternos	Rol incorrecto: 403; recurso inexistente: 404; validación de motivo; selección destacada no acreditada.
Mockups disponibles	assets/mockups/26-admin-resumen.jpg, 27-admin-tiendas.jpg, 28-admin-publicaciones.jpg, 29-admin-fundaciones.jpg

9. Integración propuesta con Spring Boot

9.1. Contratos y comunicación

La aplicación React futura consumirá la API mediante HTTPS y respuestas JSON. El cliente se configura con una base URL por entorno, transformará parámetros y DTO, y usará un único mecanismo de manejo de errores. No se duplicarán reglas del negocio del backend en el frontend.

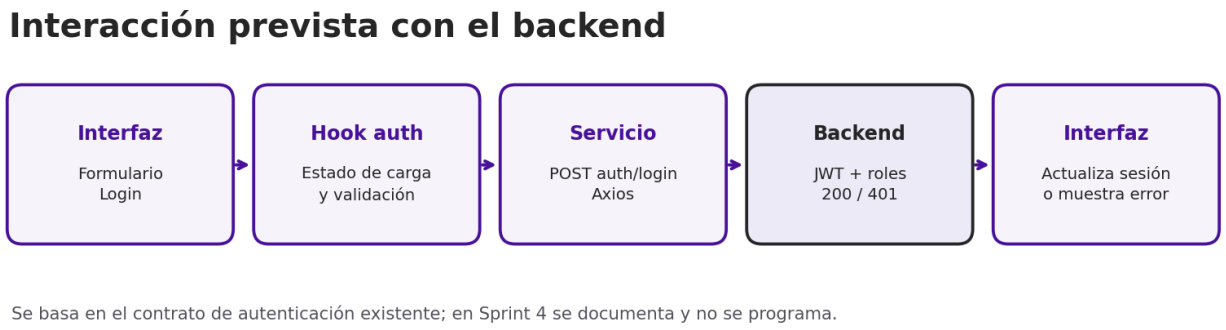


Figura 4. Secuencia de autenticación prevista para un cliente React.

9.2. Contratos reales verificables y uso previsto

Operación	Ruta real del backend (consultada)	Uso futuro del cliente
Inicio de sesión	POST /api/v1/auth/login	Enviar credenciales y recibir accessToken, tokenType, expiresIn, usuarioId y roles.
Registro comprador	POST /api/v1/auth/registro	Enviar datos de registro; respuesta 201 al crear.
Buscar catálogo	GET /api/v1/catalogo/buscar	Consultar prendas y filtros desde catálogo.
Resumen por almacén	GET /api/v1/almacenes/{nit}/catalogo/resumen	Mostrar catálogo resumido desde mapa.
Novedades	GET /api/v1/almacenes/{nit}/catalogo/novedades	Mostrar últimas novedades del local.
Distancia	GET /api/v1/almacenes/{nit}/distancia	Presentar distancia calculada.
Moderación	PATCH /api/v1/admin/catalogo/{itemId}/moderacion	Permitir decisión solo a ADMIN_STYLERADAR.

Operación	Ruta real del backend (consultada)	Uso futuro del cliente
Verificación	PATCH /api/v1/admin/almacenes/{nit}/verificacion	Registrar evaluación del almacén.
Estado usuario	PATCH /api/v1/admin/usuarios/{id}/estado	Bloqueo/inactivación/reactivación conforme permisos.
Suscripción	POST /api/v1/almacenes/{nit}/suscripciones	Vincular un plan con rol ADMIN_ALMACEN.
Renovación	PATCH /api/v1/almacenes/{nit}/suscripciones/{id}/renovacion	Cancelar renovación del siguiente período, no el vigente.

El catálogo de rutas de arriba se basó en AuthController y SecurityConfig de la rama develop del backend. Al implementar el cliente deben validarse métodos, esquema de DTO, paginación y ejemplos contra Swagger/OpenAPI de la versión vigente. La presencia de una ruta no implica que su flujo esté validado de punta a punta desde el frontend.

9.3. Sesión y token

7. El usuario inicia sesión y el cliente obtiene accessToken, expiresIn y roles desde la respuesta del backend.
8. En peticiones protegidas el cliente envía Authorization: Bearer <token> por HTTPS. No se incluyen contraseñas en URLs ni se registran tokens en consola.
9. El estado global de sesión proporciona identidad y roles para renderizar menús y rutas.
10. Ante 401 se invalida la sesión de la interfaz y se ofrece reautenticación; ante 403 se conserva la sesión pero se informa falta de permisos.
11. La estrategia de almacenamiento del token debe evaluarse por riesgo XSS y el ciclo de vida real del JWT. No se asume implementación de cookies ni refresh token: el backend actual es stateless Bearer. Se propone conservar el access token en memoria, sin credenciales ni JWT en localStorage. Una renovación con cookie HttpOnly, Secure y SameSite exigiría un contrato explícito y protección CSRF cuando aplique; no se presupone que exista. Al salir o vencer sesión se limpiarían identidad y cachés privados.

9.4. CORS, configuración y endpoints

SecurityConfig define CORS configurable mediante styleradar.cors.allowed-origins, cuyo valor predeterminado es http://localhost:3000. Antes de desplegar un frontend React se debe acordar su origen real y habilitarlo en el backend. No se debe confundir URL de la API en Azure con una URL de frontend desplegado.

9.5. Contrato de errores para presentación

HTTP	Comportamiento esperado en frontend
400	Marcar campos inválidos y explicar corrección.
401	Avisar que la sesión no es válida y permitir ingresar de nuevo.
403	Mostrar falta de permisos; no ocultar que el backend sigue imponiendo restricción.
404	Pantalla de recurso no encontrado y botón para volver al listado.
409	Mostrar conflicto y posible acción de recuperación sin afirmar éxito.
422	Exponer violación de una regla de negocio sin detalles técnicos internos.
500 / red	Explicar que hay un problema temporal; habilitar reintento.

10. Gestión de estado, validación y seguridad

10.1. Estados de interfaz

Cada consulta o formulario distinguirá idle, loading, success, empty y error. La interfaz debe mostrar retroalimentación perceptible durante peticiones, confirmar acciones importantes y prevenir duplicados durante el envío. Las respuestas paginadas conservan filtros en la navegación cuando corresponda.

10.2. Validaciones y confianza

La validación en frontend reduce errores del usuario; nunca reemplaza la del backend. Se validan formatos, obligatoriedad y coherencia básica de campos, mientras los permisos y reglas finales se verifican en Spring Boot. Las rutas privadas se presentan según roles, pero deben responder 401/403 ante intentos de acceso no autorizado.

10.3. Privacidad

La subida de fotos al probador requiere consentimiento, política de retención y tratamiento conforme al diseño final. No se debe enviar ni almacenar la foto personal en un servicio sin que esté implementado y documentado. Para el prototipo la visualización es aproximada y no demuestra integración con inteligencia artificial de virtual try-on.

11. Identidad visual y experiencia de uso

11.1. Paleta oficial

Color	HEX	Uso en la interfaz
Violeta	#48119A	Marca, acción principal y elementos activos.
Grafito	#262626	Fondos oscuros, contraste y elegancia.
Blanco	#FFFFFF	Superficies y texto contrastante.
Arena	#E8DCCB	Paneles y superficies cálidas.
Lavanda	#B9B9CA	Estados y acentos sobre fondos oscuros.

La identidad propone una proporción visual 60 % neutros, 30 % secundarios y 10 % violeta. Las combinaciones y ratios indicados en el README son blanco/grafito 15,13:1, lavanda/grafito 7,82:1, blanco/violeta 11,52:1, arena/violeta 8,52:1 y violeta/blanco 11,5:1. Esas cifras se citan tal como están documentadas en el repositorio, sin auditoría independiente de contraste.

11.2. Tipografía, componentes y símbolos

La fuente corporativa es Poppins, con jerarquías documentadas en el manual: título principal Black 72, secundario Bold 44, terciario SemiBold 24, texto Regular 16, etiqueta Medium 11 y ayuda Light 13. Los botones tienen esquinas de 12 px, y los iconos de línea declarada de 1,6 px. Las opciones de un menú se distinguen por marca de selección además de color.

11.3. Mascota y referentes

La identidad utiliza cinco amigos de Bogotá: Antonia (sastre), Mateo (calle), Aura (futuro), Nora (rebelde) y Tomás (esencial). Deben integrarse de manera funcional en tutoriales y estados vacíos o de error, siguiendo el manual de identidad; no usarlos como adorno repetido. La colección de imágenes está en assets/mascota/.

11.4. Responsive y accesibilidad

- El README del prototipo HTML documenta pruebas a 390 px y 1440 px; conservar un diseño sin desplazamiento horizontal.
- En móvil, ofrecer menú accesible y tarjetas en una columna cuando proceda.
- Todas las acciones navegables por teclado y visibles al recibir foco; no depender únicamente del color.
- Mensajes de validación asociados a campo, textos alternativos para fotografías e iconos y nombres accesibles para botones.
- La guía del repositorio declara WCAG 2.1 AA como objetivo; repetir verificación real antes de considerar el frontend productivo.

12. Trazabilidad, verificación y limitaciones

12.1. Ejemplos de trazabilidad UI → RF → API

RF	Ubicación en prototipo	Evidencia backend / observación
RF02	Inicio de sesión	POST /api/v1/auth/login, JWT y roles.
RF05–RF06	Mi cuenta → Recomendaciones	Feed y recomendación en API; documentar contrato en integración.
RF17–RF21	Mapa / Tiendas en el radar	Marcadores, filtro, resumen, novedades y distancia.
RF29–RF32	Mis búsquedas y alertas / campana	Falló creación positiva en Swagger local por HTTP 409; no declarar cierre E2E.
RF61–RF62	Panel administrador → Publicaciones / Tiendas	Moderación y verificación documentadas en backend.
RF66	Panel administrador → Usuarios	Cambiar estado con rol administrador.
RF67–RF68	Mi tienda → Suscripción	Alta de suscripción y cancelación de renovación.
RF72	Playlists / panel administrativo	Ranking disponible; la selección para destacar según HU requiere validación.

12.2. Estrategia de validación documental

12. Confirmar en cada mockup el happy path, el error y el estado vacío correspondiente.
13. Revisar que pantalla, actor, RF y contrato HTTP utilicen denominaciones idénticas.
14. Comprobar los accesos del prototipo para invitados, comprador, almacén, fundación y administrador.
15. Registrar capturas de diseño y referenciar el archivo real styleradar-frontend-v3/prototipo/index.html. El prototipo HTML es navegable localmente, sin depender de Figma y sin llamadas al backend.
16. Cuando exista código React, añadir tests de rutas, componentes y servicios con API simulada y pruebas E2E contra entorno autorizado.

12.3. Limitaciones y aspectos que requieren confirmación

Tema	Situación evidenciada / acción requerida
Implementación React	No está implementada; la guía de Sprint 4 solicita documentar el frontend sin desarrollarlo.
Prototipo HTML	Existe en prototipo/index.html; simula datos locales. Alinear su permanencia en repo con la regla de entrega “solo README e imágenes”.
Prototipo navegable / requisito Figma	No existe archivo ni enlace Figma del equipo. Prototipo disponible en https://github.com/JUANNICOLASA/Frontend-Style-Radar/tree/main/styleradar-frontend-v3/prototipo . El archivo index.html se ejecuta en el navegador local y no está conectado a Spring Boot. Solicitar al profesor la aceptación de un prototipo HTML como equivalente al enlace Figma requerido.
Identidad / manual	El enlace al manual fue corregido en el README: docs/StyleRadar_Manual_de_Identidadv2.html. Se verificó la existencia del archivo en esta revisión documental.
Requerimientos 75 RF	Se cotejaron las responsabilidades de los módulos con los 75 RF del Excel v3 y las rutas del HTML. Esto valida la correspondencia documental; la ejecución funcional y los contratos HTTP siguen pendientes de integración.
Mascotas y RF73–RF75	En docs/sprints/sprint-3-y-4.md aparecen tareas de prototipo pendientes; no dar por implementadas sin revisión.
Alertas RF29–RF32	La evidencia disponible reportó respuesta 409/SQLState 23514 al crear alerta: deuda que afecta demo.
RF72 destacado	El ranking no acredita la selección administrativa de playlists destacadas que exige la HU.
Despliegue frontend	No se identificó URL de React implementado; el prototipo local actúa como demostrador.

12.4. Relación de recursos visuales existentes

El repositorio dispone de mockups por funcionalidad bajo styleradar-frontend-v3/assets/mockups/ (01-portada, 02-login, 03-registro, 10-catalogo, 11-mapa, 14-probador, 15-playlist, 18-perfil, 21-mi-tienda, 24-mi-fundacion y 26-admin-resumen, entre otros). Estos son los referentes visuales oficiales para insertar capturas o revisar pantallas. Los diagramas de este documento representan arquitectura propuesta y no reemplazan los mockups. El demostrador navegable está en styleradar-frontend-v3/prototipo/index.html; no existe un diseño independiente en Figma.

13. Referencias documentales

[1] Escuela Colombiana de Ingeniería Julio Garavito. DOSW 1, Semana 11: arquitectura frontend, React, Axios y High Level Design. Diapositivas aportadas al proyecto.

[2] Escuela Colombiana de Ingeniería Julio Garavito. Sprint #4 — Final · DOSW. Documento HTML entregado para el proyecto, 2026.

[3] JUANNICOLASA. Frontend-Style-Radar. Repositorio GitHub: https://github.com/JUANNICOLASA/Frontend-Style-Radar

[4] JUANNICOLASA. styleradar-frontend-v3/README.md (rama docs/sprint4-arquitectura-frontend, revisión documental 1.1, octubre 2026): identidad, actores, pantallas y comportamiento del prototipo heredado de main.

[5] JUANNICOLASA. styleradar-frontend-v3/docs/sprints/sprint-3-y-4.md (situación de pendientes y entregables de interfaz).

[6] CatrachoINXS. StyleRadar-Backend, rama develop: Rol.java, AuthController.java, LoginResponseDTO.java, SecurityConfig.java. https://github.com/CatrachoINXS/StyleRadar-Backend

[7] React Team. Documentación oficial React: https://react.dev/

[8] React Router. Documentación: https://reactrouter.com/

[9] Axios. Documentación: https://axios-http.com/

[10] W3C. Web Content Accessibility Guidelines (WCAG): https://www.w3.org/WAI/standards-guidelines/wcag/

Historial de revisiones

Versión	Fecha	Descripción	Estado
1.1	18 de octubre de 2026	Versión ajustada al prototipo HTML real del repositorio; Figma no existe en el proyecto y se declara la diferencia frente a la guía del Sprint 4. Revisión final: versión 1.1 unificada en portada, control, encabezados, pies y metadatos; módulos y RF alineados con README y HTML; PDF regenerado desde el Word, sin retirar contenido ni diagramas.	Revisión documental final; propuesta pendiente de implementación

ANEXO A | DIAGRAMAS DE NAVEGACIÓN POR ROL

Fuente de los recorridos: prototipo navegable HTML v3, README y mockups del repositorio Frontend-Style-Radar. Los diagramas describen el comportamiento del prototipo; no equivalen a funcionalidades ya conectadas a la API.

Convención visual: violeta = inicio / entrada; lavanda = pantallas principales; verde = resultado exitoso; amarillo = decisión o verificación; rosa = error o estado vacío. Las flechas punteadas representan caminos alternativos o de error.

A.1. Visitante no autenticado

Pantallas públicas, descubrimiento y redirección a inicio de sesión para acciones personales.

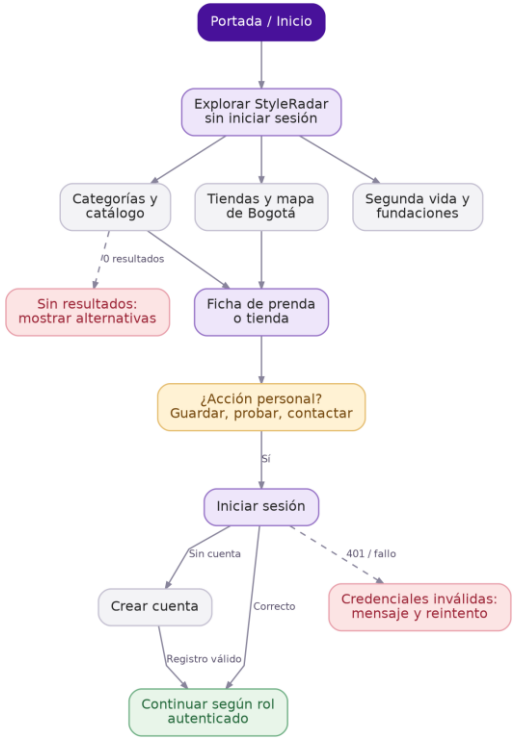


Figura A.1. Diagrama de navegación: Visitante no autenticado. Diseño documentado para Sprint 4.

A.2. Comprador (COMPRADOR)

Recorrido desde autenticación hacia búsqueda, recomendaciones, listas, contacto y segunda vida.

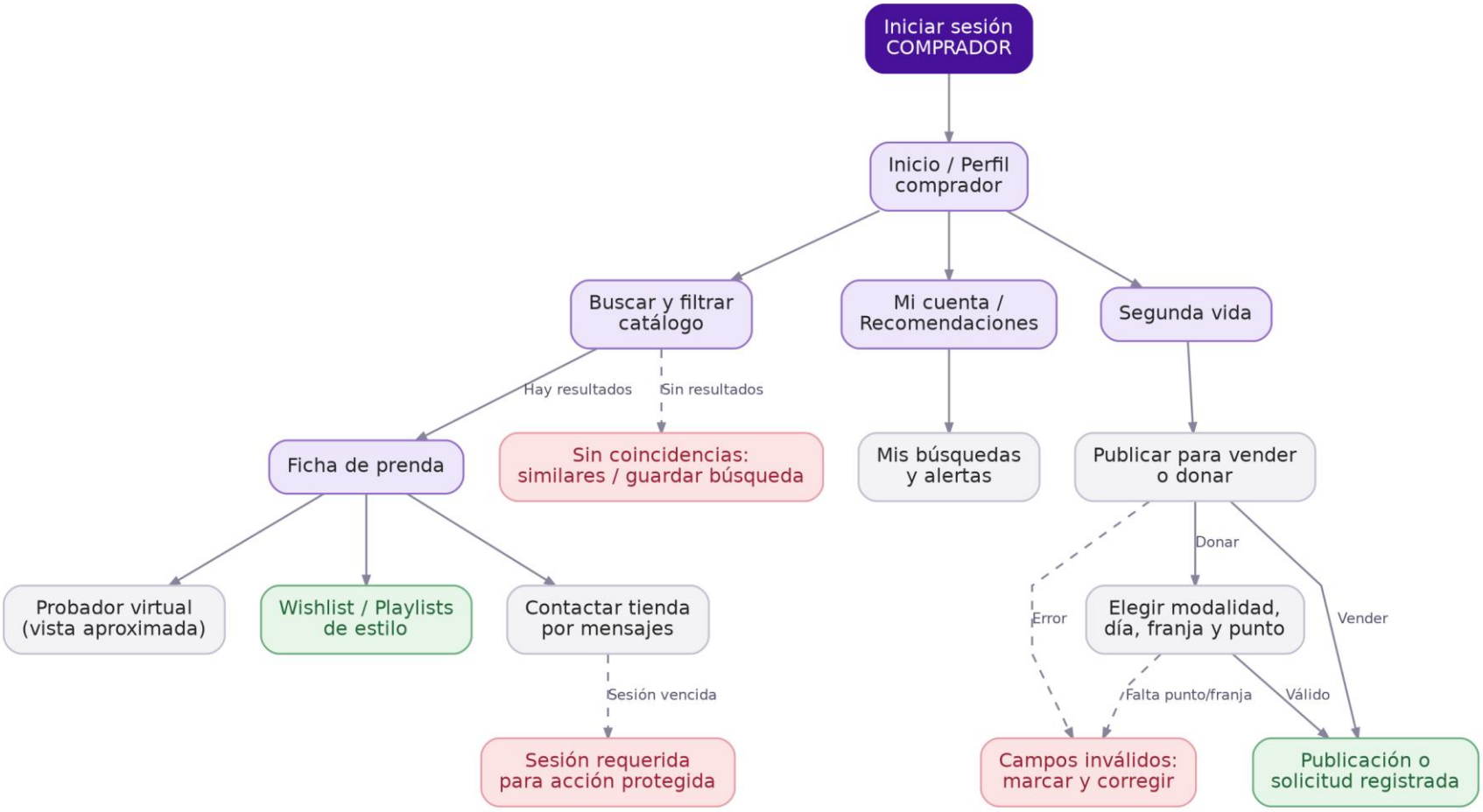


Figura A.2. Diagrama de navegación: Comprador (COMPRADOR). Diseño documentado para Sprint 4.

A.3. Administrador de almacén (ADMIN_ALMACEN)

Inventario, publicación sujeta a moderación, estadísticas, clientes y renovación de suscripción.

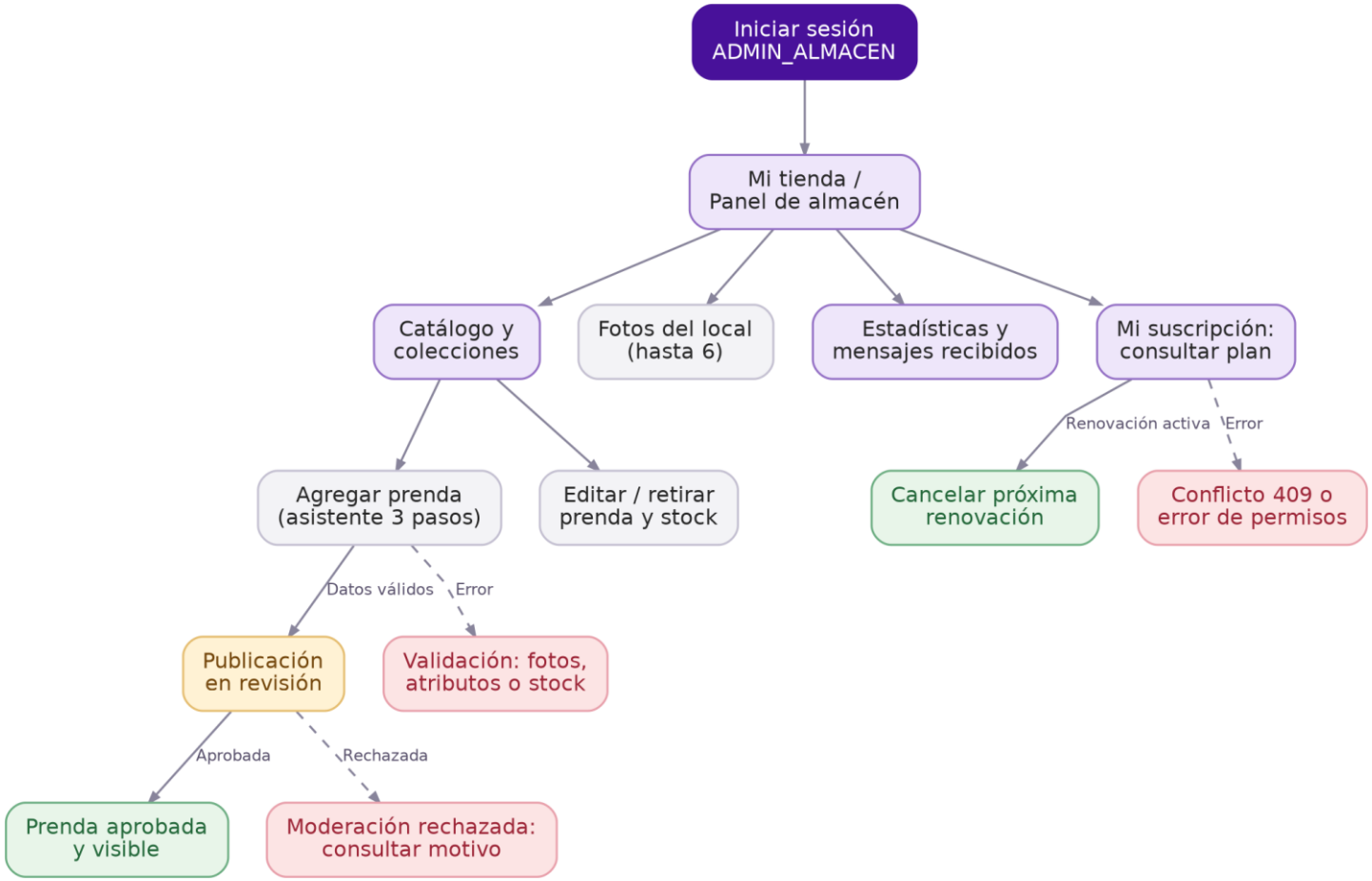


Figura A.3. Diagrama de navegación: Administrador de almacén (ADMIN_ALMACEN). Diseño documentado para Sprint 4.

A.4. Representante de fundación (REPRESENTANTE_FUNDACION)

Perfil de fundación y tramitación de propuestas de donación con punto y franja indicados por el donante.

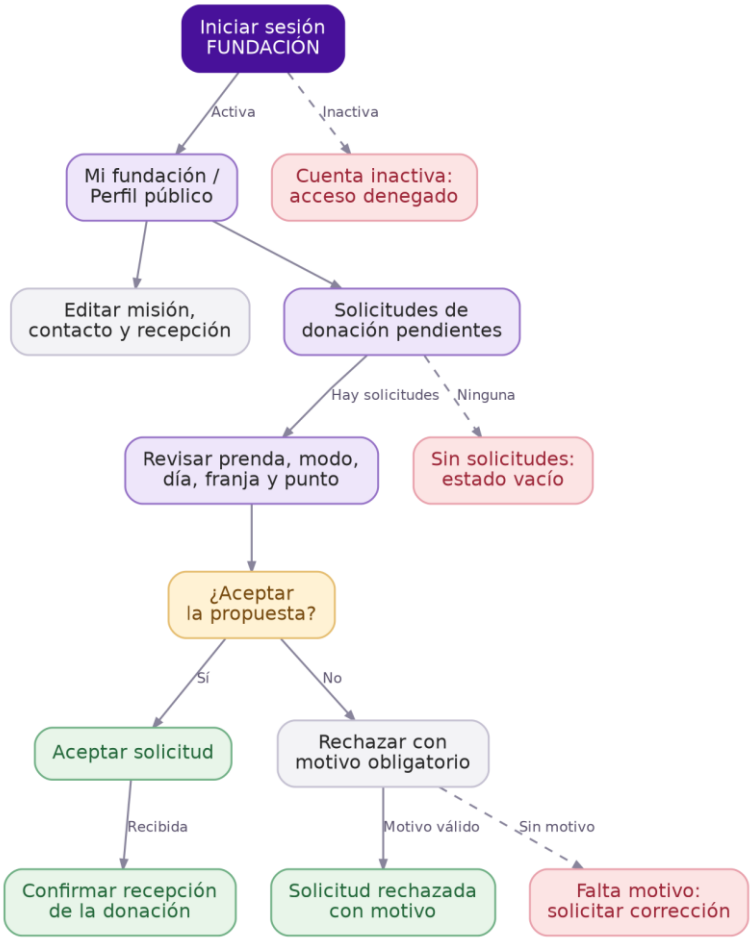


Figura A.4. Diagrama de navegación: Representante de fundación (REPRESENTANTE_FUNDACION). Diseño documentado para Sprint 4.

A.5. Administrador StyleRadar (ADMIN_STYLERADAR)

Supervisión, moderación, verificación, usuarios y otras secciones de la interfaz administrativa.

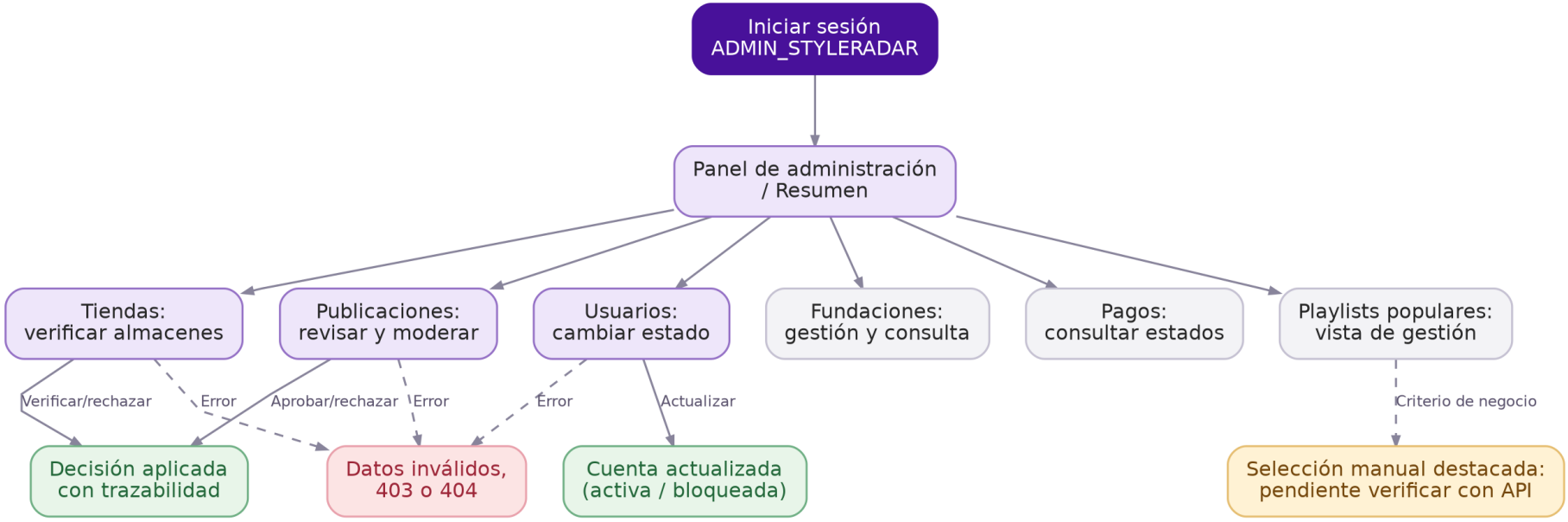


Figura A.5. Diagrama de navegación: Administrador StyleRadar (ADMIN_STYLERADAR). Diseño documentado para Sprint 4.

Notas de alcance

- El visitante sin sesión puede navegar por recursos públicos. Guardar, publicar y contactar requieren iniciar sesión, según el flujo del prototipo y los permisos del backend.
- El probador virtual del prototipo presenta visualización aproximada; no se acredita un motor de IA de prueba virtual en el backend.
- La moderación se refleja en la visibilidad del catálogo; publicar una prenda no equivale a aprobarla automáticamente.
- La fundación recibe la modalidad, día, franja y punto propuestos por quien dona; aceptar o rechazar no sustituye esa propuesta.
- La selección administrativa de playlists destacadas aparece en el prototipo, pero la API integrada fue revisada previamente como ranking por popularidad; esa discrepancia debe declararse hasta su resolución.
- Estos diagramas no reemplazan pruebas de ejecución ni demuestran conexión React–Spring Boot. El frontend React nivel 3 es una arquitectura propuesta para una implementación futura.